# JAVA COLLECTIONS CASE STUDY 5

**Student Name:** Aabhas Katiyar
**Roll Number:** 2500290130001

## Attendance System - LinkedHashSet

**Topic:** LinkedHashSet & Insertion-Order Preservation
**Environment:** Java SE 17+ / 24

## 1. Problem Statement & Scenario Description

**Scenario:** A school wants to record the IDs of students who attended an event. A student should be recorded only once, and the order in which students arrived must be preserved.

| # | Required Operation / Method | Implementation Details & Collections Logic |
|---|---|---|
| 1 | **markAttendance(Set attendance, int studentId)** | Inserts student ID into LinkedHashSet; duplicate arrivals are safely ignored. |
| 2 | **isPresent(Set attendance, int studentId)** | Determines if the student ID exists in the attendance record via contains(). |
| 3 | **getAttendanceCount(Set attendance)** | Returns total count of unique attendees present at the event. |
| 4 | **displayAttendance(Set attendance)** | Prints full attendee sequence formatted in the exact order of their arrival. |

## 2. Collections Framework Concepts & Technical Analysis

- **LinkedHashSet Architecture:** Combines a hash table with a running doubly-linked list to achieve both uniqueness and insertion-order preservation.
- **Chronological Order Preservation:** Unlike HashSet, iterating over LinkedHashSet visits elements in the exact order they were inserted.
- **Event Attendance Mechanics:** First arrival determines the student position in the log; subsequent arrivals do not alter order or count.

## 3. Complete Java Solution Code (Human Written - No Comments)

Clean, idiomatic Java implementation completed according to instructions with zero comments.

**Source File: src/collections_case_studies/AttendanceSystem.java**

```
package collections_case_studies;

import java.util.LinkedHashSet;
import java.util.Set;

public class AttendanceSystem {

    public static void markAttendance(Set<Integer> attendance, int studentId) {
        attendance.add(studentId);
    }

    public static boolean isPresent(Set<Integer> attendance, int studentId) {
        return attendance.contains(studentId);
    }

    public static int getAttendanceCount(Set<Integer> attendance) {
        return attendance.size();
    }

    public static void displayAttendance(Set<Integer> attendance) {
        System.out.println("Attendance List: " + attendance);
    }

    public static void main(String[] args) {
        Set<Integer> attendance = new LinkedHashSet<>();

        markAttendance(attendance, 105);
        markAttendance(attendance, 102);
        markAttendance(attendance, 108);
        markAttendance(attendance, 105);
        markAttendance(attendance, 101);

        displayAttendance(attendance);

        System.out.println("Student 108 present: " + isPresent(attendance, 108));
        System.out.println("Total Present: " + getAttendanceCount(attendance));
    }
}
```

## 4. Program Execution & Verification Output

Console output produced by running the completed driver code matching expected laboratory output:

**Console Standard Output (stdout): java collections_case_studies.AttendanceSystem**

```
Attendance List: [105, 102, 108, 101]
Student 108 present: true
Total Present: 4
```